ARCHITECTURE AND PROTOCOLS

1. SCREENS MAP

```
Splash
  └─ Login / Register (Firebase Auth)
        └─ Main Shell (Bottom Navigation)
              ├─ Subscriptions List
              │     ├─ Subscription Detail ── Edit Subscription
              │     └─ Add Subscription ── Service Catalog (JSON)
              ├─ Groups
              │     ├─ Group Detail ── Members and Payment Status
              │     └─ Create Group / Invite Members
              ├─ Statistics (animated charts)
              └─ Settings (currency, notification days, backup/sync, logout)
```

2. DATA MODEL

The application uses four main data classes. Subscription is the main class, and Group and PaymentRecord are used for shared subscriptions.

Table 1: Subscription

Field	Type	Note
id	String	UUID, also the Firestore document id
name	String	e.g. Netflix
price	double	Total cost of the plan per billing cycle
currency	String	TRY, USD, EUR
billingCycle	enum	monthly, yearly, weekly
nextRenewalDate	DateTime	used for notifications
category	enum	entertainment, music, education, software, other
type	enum	personal, shared
groupId	String?	only for shared subscriptions
logoAsset	String?	from the JSON catalog
isActive	bool	
updatedAt	DateTime	used to resolve sync conflicts

Table 2: Group

Field	Type	Note
id	String	UUID
name	String	e.g. Flat mates
ownerId	String	primary account holder (user id)
memberIds	List<String>	user ids; the share is price / number of members

Table 3: PaymentRecord

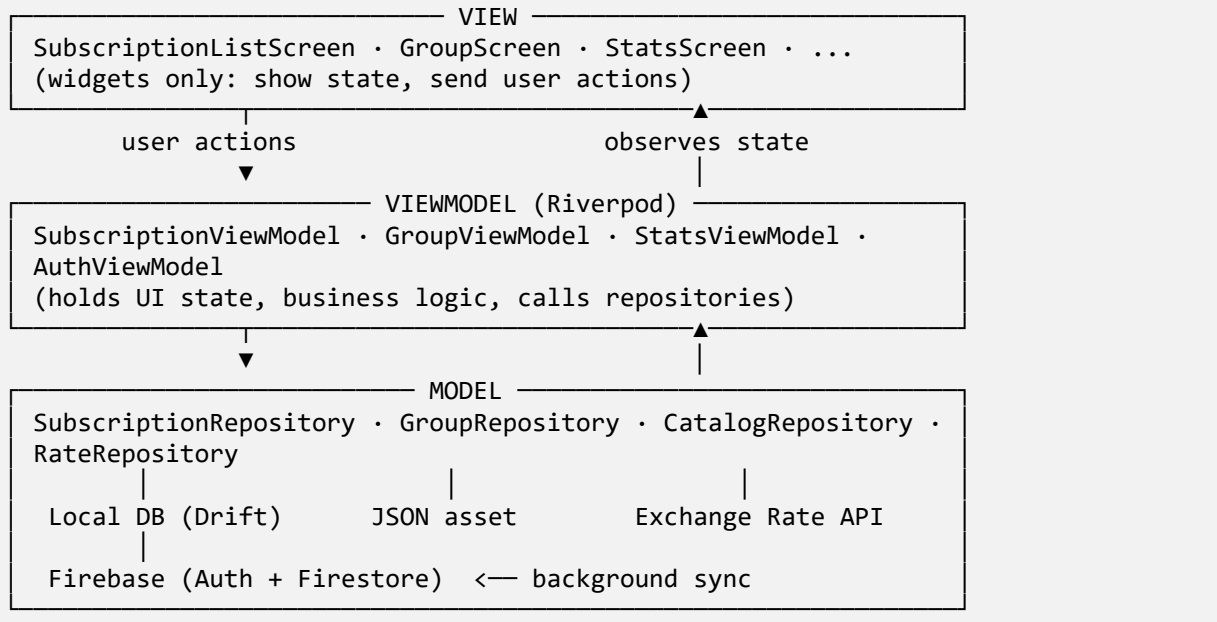
Field	Type	Note
id	String	UUID
subscriptionId	String	which shared subscription
memberId	String	who owes the share
period	String	billing month, e.g. 2026-11
shareAmount	double	C_i / N_i from the budget model
status	enum	paid, pending
paidAt	DateTime?	set when the member pays

CatalogService (from JSON): name, logo, averagePrice, currency, category

ExchangeRate (from API): baseCurrency, rates (Map<String, double>), fetchedAt

Relations: one user can create many Groups, and one Group has many members. A shared Subscription belongs to one Group and has one PaymentRecord for each member in each billing month. CatalogService only pre-fills the add form and has no direct relation to the stored data.

3. MVVM DIAGRAM



Data flow: the View never talks to the data sources directly. It calls the ViewModel, the ViewModel calls the Repository, and the Repository decides whether to use the local DB, the API or Firebase.

4. PROVIDER / RIVERPOD CHOICE

We chose Riverpod (flutter_riverpod) instead of Provider for state management. The reasons are:

- Compile-time safety: providers do not depend on BuildContext, so there are fewer runtime errors than with Provider.
- Built-in async support: AsyncNotifier and FutureProvider handle loading/error/data states, which we need for the exchange rate API and Firebase calls.
- Easy testing: providers can be overridden, so repositories can be replaced with fakes in tests.
- Fits MVVM: Notifier / AsyncNotifier classes act directly as ViewModels, and repositories are injected through providers.
- Scalability: dependencies between providers (for example, Stats depends on the Subscription list and Exchange Rates) are declared clearly and update automatically.

5. KEY PROTOCOLS AND REPOSITORY INTERFACES

To maintain a clean separation of concerns within our MVVM architecture, the ViewModel layer communicates with the Model layer strictly through abstract repository interfaces and standard communication protocols:

- Repository Interfaces (Data Abstraction):

- SubscriptionRepository: Defines the interface for adding, editing, deleting, and fetching personal and shared subscriptions from both local storage (Drift) and Cloud Firestore.
- GroupRepository: Manages group creation, member invitations, and shared PaymentRecord statuses (paid/pending).
- CatalogRepository: Reads and parses the local bundled JSON asset to pre-fill popular subscription services.
- RateRepository: Fetches live currency conversion rates (USD/EUR to TRY) and caches the latest results.

- Communication & Persistence Protocols:

- HTTPS / REST over JSON: Used by RateRepository to securely fetch real-time exchange rates from the external Exchange Rate API.
- Firebase SDK (gRPC / WebSockets): Used by AuthViewModel (Firebase Auth) for secure user authentication and by Cloud Firestore for real-time background synchronization of group payment statuses when the device is online.
- SQLite (via Drift ORM): Used as the primary local persistence protocol to guarantee offline-first read/write access and resolve synchronization conflicts using the updatedAt timestamp.